WealthWise

Software Design Document

Mutual Fund & SIP Tracking Platform

Field	Details
Document Type	Software Design Document (SDD)
Project Name	WealthWise
Architecture	Monolithic Layered Architecture
Tech Stack	React + Java Spring Boot + MySQL
Prepared By	Team-09
Year	2026

1. Project Overview

Attribute	Value
Project Name	WealthWise
Type	Web Application
Architecture	Monolithic Layered Architecture
Frontend	React (JavaScript)
Backend	Java Spring Boot
Database	MySQL
IDE	Eclipse
External API	MFAPI (api.mfapi.in) — Live NAV data
Auth	JWT (JSON Web Token) + BCrypt

2. Problem Statement

Individual investors who invest in mutual funds face the following real-world problems:

1. No single place to track all their SIP investments across multiple funds and platforms.
2. Investors forget SIP due dates and miss payments without any reminder.
3. Cannot easily see current portfolio value and actual returns (profit or loss).
4. No visibility on how each individual fund is performing.
5. No simple way to browse available mutual funds with risk and category filters.
6. Admin has no dedicated tool to manage investors and send alerts at scale.

3. Proposed Solution

WealthWise is a web-based mutual fund and SIP tracking platform that solves these problems through two role-based interfaces:

For Investors:

- Dashboard showing total invested, current value, profit/loss and active SIPs at a glance.
- Browse and invest in mutual funds using SIP or Lumpsum investment type.
- Portfolio page with real-time returns calculated using live NAV from MFAPI.
- SIP tracking with complete history, missed payment records and upcoming due dates.
- In-app notification bell for alerts sent by admin.

For Admin:

- Manage AMCs and mutual funds with category, risk level and scheme code.
- View and manage all investor accounts with activate or deactivate control.
- Send automated alerts (SIP due reminder, missed SIP, SIP completion) using templates.
- Send general announcements to all investors with one click.
- Dashboard showing platform-wide statistics including total users, active SIPs and total AUM.

4. Functional Requirements

4.1 Admin Requirements

ID	Requirement
FR1	Admin logs in using a pre-configured secret key stored in the database.
FR2	Admin can add and view all AMCs (Asset Management Companies).
FR3	Admin can add mutual funds with fund name, category, risk level, AMC and scheme code.
FR4	Admin can view all registered investors with name, email, phone, joined date and total SIPs.
FR5	Admin can activate or deactivate any investor account.
FR6	Admin can view individual investor full profile and their SIP list.
FR7	Admin can send automated SIP due reminder alerts to all relevant investors.
FR8	Admin can send missed SIP alerts to investors who missed payments.
FR9	Admin can send SIP completion alerts to investors whose SIP tenure ended.
FR10	Admin can send general announcements to all investors at once.
FR11	Admin dashboard shows total users, active SIPs, listed funds and total AUM.

4.2 Investor Requirements

ID	Requirement
FR12	Investor can self-register with name, email, phone and password.
FR13	Investor can log in and receive a JWT token for session management.
FR14	Investor can browse all available mutual funds listed by the admin.
FR15	Investor can filter funds by category (Equity, Debt, Hybrid) and risk level.
FR16	Investor can invest in a fund via SIP or Lumpsum investment type.
FR17	Units are auto-calculated from NAV at the time of investment.
FR18	Investor can view portfolio with total units, current NAV, invested amount, current value, profit/loss and return percentage.
FR19	Investor can view all active SIPs with fund name, amount, start date, next due date and status.
FR20	Investor can view SIP transaction history showing paid, missed and pending records.
FR21	Investor receives and reads in-app notifications from admin.
FR22	Investor can mark notifications as read via the bell icon.

5. Non-Functional Requirements

ID	Category	Requirement
NFR1	Security	Passwords stored as BCrypt hash. APIs protected using JWT tokens. Role-based access control for ADMIN and INVESTOR.
NFR2	Performance	Portfolio calculations done at runtime using live NAV. No redundant computed data stored in database.
NFR3	Reliability	NAV fetched daily via Spring @Scheduled job from MFAPI free public API. Failures are logged and skipped gracefully.
NFR4	Usability	Separate login flows for admin (secret key) and investor (email + password). Clean React UI with role-based dashboard.
NFR5	Scalability	Monolithic architecture for current scope. Package structure designed to support microservices migration in future.
NFR6	Maintainability	Layered architecture (Controller, Service, Repository, Model) ensures separation of concerns and easy debugging.

6. System Architecture

WealthWise follows a Monolithic Layered Architecture with four distinct layers inside a single Spring Boot application.

6.1 Architecture Diagram

```
+-----------------------------------------+
|          React Frontend                 |
|   Investor Portal    |    Admin Panel    |
+----------------+-----+-------------------+
                 |  HTTP REST API (JSON)
                 v
+-----------------------------------------+
|       Java Spring Boot Backend          |
|                                         |
|  Controller Layer  (REST Endpoints)     |
|          |                              |
|  Service Layer     (Business Logic)     |
|          |                              |
|  Repository Layer  (JPA / SQL Queries)  |
|          |                              |
|  Model Layer       (Entities / Tables)  |
+----------------+------------------------+
                 |
                 v
+-----------------------------------------+
|          MySQL Database                 |
|          9 Tables                       |
+-----------------------------------------+
                 |
                 v  (Daily Scheduler)
+-----------------------------------------+
|   MFAPI  (api.mfapi.in)                 |
|   Free Live NAV Data — No API Key Needed |
+-----------------------------------------+
```

6.2 Layer Responsibilities

Layer	Technology	Responsibility
Controller	Spring MVC (@RestController)	Receives HTTP requests, validates input, returns JSON responses.
Service	Spring (@Service)	Contains all business logic, calculations and orchestration.
Repository	Spring Data JPA (@Repository)	Database queries using JPA methods and custom JPQL.
Model	JPA Entities (@Entity)	Maps Java classes to MySQL tables.
Security	Spring Security + JWT	Authenticates requests, enforces role-based access.
Scheduler	Spring @Scheduled	Runs daily NAV fetch job and monthly SIP transaction creation.

7. Project Structure

```
wealthwise-backend/
|
+-- src/main/java/com/wealthwise/
|   |
|   +-- WealthwiseApplication.java        (main class)
|   |
|   +-- config/
|   |   +-- SecurityConfig.java           (Spring Security setup)
|   |   +-- CorsConfig.java               (allow React on localhost:3000)
|   |
|   +-- model/                            (9 DB entity classes)
|   |   +-- User.java
|   |   +-- Amc.java
|   |   +-- MutualFund.java
|   |   +-- NavHistory.java
|   |   +-- Investment.java
|   |   +-- Sip.java
|   |   +-- SipTransaction.java
|   |   +-- Portfolio.java
|   |   +-- Alert.java
|   |
|   +-- repository/                       (JPA interfaces)
|   |   +-- UserRepository.java
|   |   +-- AmcRepository.java
|   |   +-- MutualFundRepository.java
|   |   +-- NavHistoryRepository.java
|   |   +-- InvestmentRepository.java
|   |   +-- SipRepository.java
|   |   +-- SipTransactionRepository.java
|   |   +-- PortfolioRepository.java
|   |   +-- AlertRepository.java
|   |
|   +-- service/                          (business logic)
|   |   +-- AuthService.java
|   |   +-- AmcService.java
|   |   +-- MutualFundService.java
|   |   +-- NavService.java               (MFAPI scheduler)
|   |   +-- InvestmentService.java
|   |   +-- SipService.java
|   |   +-- SipTransactionService.java
|   |   +-- PortfolioService.java
|   |   +-- AlertService.java
|   |
|   +-- controller/                       (REST APIs)
|   |   +-- AuthController.java
|   |   +-- AmcController.java
|   |   +-- MutualFundController.java
|   |   +-- InvestmentController.java
|   |   +-- SipController.java
|   |   +-- PortfolioController.java
|   |   +-- AlertController.java
|   |
|   +-- dto/
|   |   +-- request/   (LoginRequest, RegisterRequest, etc.)
|   |   +-- response/  (AuthResponse, PortfolioResponse, etc.)
|   |
|   +-- security/
|   |   +-- JwtUtil.java
|   |   +-- JwtFilter.java
|   |   +-- UserDetailsServiceImpl.java
|   |
|   +-- exception/
|       +-- GlobalExceptionHandler.java
```

```
|       +-- ResourceNotFoundException.java
|
+-- src/main/resources/
|   +-- application.properties
|
+-- pom.xml
```

8. Module Breakdown

MODULE 1 Week 1	**Authentication Module** Handles investor self-registration, investor login and admin login via secret key. Generates and validates JWT tokens. Enforces role-based access across all APIs.
MODULE 2 Week 2	**Fund Management Module** Admin adds and manages AMCs and Mutual Funds. Daily scheduled job fetches live NAV from MFAPI and updates the database. Investors can browse and filter available funds.
MODULE 3 Week 3	**Investment Management Module** Investor adds SIP or Lumpsum investment. Units are auto-calculated from current NAV. Portfolio table is created or updated after every investment. Returns calculated at runtime.
MODULE 4 Week 4	**SIP Management Module** Investor views active SIPs, SIP transaction history and missed payments. Monthly scheduler creates PENDING transaction rows. Status updated to PAID or MISSED based on payment date.
MODULE 5 Week 5	**Admin Dashboard & User Management** Admin views platform statistics (total users, active SIPs, listed funds, total AUM). Admin manages investor accounts including view profile, view SIPs and activate/deactivate.
MODULE 6 Week 6	**Alert System Module** Admin sends automated template-based alerts (SIP due, missed SIP, SIP completion) with one click. Admin sends general announcements to all investors. Investors see notifications via bell icon.
MODULE 7 Week 7	**Investor Dashboard Module** Aggregates all investor data into a single dashboard API. Shows total invested, current value, profit/loss, return percentage, portfolio summary and upcoming SIP due dates.
MODULE 8 Week 8	**Testing & Final Polish** End-to-end testing of all modules and flows. Bug fixes, edge case handling, UI polish and demo preparation with sample data.

9. API Endpoints

9.1 Authentication APIs

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Investor self-registration
POST	/api/auth/login	Public	Investor login — returns JWT token
POST	/api/auth/admin/login	Public	Admin login via secret key — returns JWT token

9.2 Fund Management APIs

Method	Endpoint	Access	Description
POST	/api/amc	Admin	Add a new AMC
GET	/api/amc	Admin	Get all AMCs
POST	/api/funds	Admin	Add a new mutual fund
GET	/api/funds	Investor	Browse all funds (filter by category, risk)
GET	/api/funds/{id}	Investor	Get single fund details and current NAV

9.3 Investment & Portfolio APIs

Method	Endpoint	Access	Description
POST	/api/investments	Investor	Add new SIP or Lumpsum investment
GET	/api/investments	Investor	Get all investment transactions
GET	/api/portfolio	Investor	Get portfolio with live returns

9.4 SIP APIs

Method	Endpoint	Access	Description
POST	/api/sips	Investor	Create a new SIP plan
GET	/api/sips	Investor	Get all active SIPs
GET	/api/sips/transactions	Investor	SIP history (paid, missed, pending)
GET	/api/sips/missed	Investor	Get only missed SIP transactions

9.5 Admin & Alert APIs

Method	Endpoint	Access	Description
GET	/api/admin/dashboard	Admin	Platform stats (users, SIPs, funds, AUM)
GET	/api/users	Admin	All investor accounts
GET	/api/users/{id}	Admin	Investor profile
GET	/api/users/{id}/sips	Admin	Investor SIP list
PUT	/api/users/{id}/status	Admin	Activate or deactivate investor
POST	/api/alerts/send/sip-due	Admin	Send SIP due reminders to all relevant investors
POST	/api/alerts/send/missed-sip	Admin	Send missed SIP alerts
POST	/api/alerts/send/completion	Admin	Send SIP completion alerts
POST	/api/alerts/send/general	Admin	Send general message to all investors
GET	/api/alerts/my	Investor	Get my notifications
PUT	/api/alerts/{id}/read	Investor	Mark notification as read
GET	/api/alerts/unread-count	Investor	Get unread notifications count

10. Security Design

Aspect	Implementation
Authentication	JWT (JSON Web Token) — token generated on login, sent in Authorization header for every request.
Password Storage	BCrypt hashing — passwords never stored in plain text.
Admin Access	Admin uses a secret key (not email/password) — no self-registration possible for admin.
Route Protection	@PreAuthorize(hasRole) on all admin and investor APIs.
Token Contents	Email, role (ADMIN or INVESTOR), issued at, expiry (24 hours).
CORS	Configured in CorsConfig.java to allow React frontend on localhost:3000.
Token Expiry	Token expires after 24 hours — investor must log in again.

11. Database Design Summary

#	Table	Purpose
1	users	Authentication for admin and investor. Stores role, status and admin_key.
2	amc	Asset Management Companies added by admin.
3	mutual_funds	Fund details including category, risk level, live NAV and MFAPI scheme code.
4	nav_history	Daily NAV records per fund fetched from MFAPI.
5	investments	Every investment transaction (SIP installment or lumpsum).
6	sips	SIP plans created by investors — frequency, dates and status.
7	sip_transactions	Monthly installment records tracking PAID, MISSED and PENDING status.
8	portfolio	Summary of holdings per investor per fund — total units and total invested.
9	alerts	In-app notifications sent by admin to investors.

11.1 Calculated Fields (Never Stored in DB)

Field	Formula	Used On
units_purchased	amount / nav_at_investment	Add Investment
current_value	total_units x current_nav	Portfolio, Dashboard
profit_loss	current_value - total_invested	Portfolio, Dashboard
return %	(profit_loss / total_invested) x 100	Portfolio, Dashboard
total_aum	SUM of all portfolio.total_invested	Admin Dashboard
next_due_date	previous_due_date + frequency (1 or 3 months)	SIPs, Dashboard

12. External API Integration — MFAPI

Attribute	Details
API Name	MFAPI
URL	https://api.mfapi.in
Purpose	Fetch live daily NAV for all listed mutual funds
Cost	Free — no API key required
Trigger	Spring @Scheduled job — runs daily at 8:00 PM
Endpoint	GET https://api.mfapi.in/mf/{scheme_code}
Search	GET https://api.mfapi.in/mf/search?q=fund+name

12.1 NAV Fetch Flow

```
Step 1: Admin adds fund with scheme_code from MFAPI
Step 2: @Scheduled job runs daily at 8 PM
Step 3: For each fund — call api.mfapi.in/mf/{scheme_code}
Step 4: Extract latest NAV and date from response
Step 5: Update current_nav in mutual_funds table
Step 6: Insert row in nav_history (fund_id, nav_price, nav_date)
Step 7: Portfolio current_value recalculated at runtime on next request
```

13. Development Timeline

Week	Module	Backend Tasks	Frontend Tasks
Week 1	Setup + Auth	Project setup, DB creation, User entity, JWT, Register/Login APIs	Login page, Register page, JWT storage, role-based redirect
Week 2	Fund Management	AMC entity + APIs, MutualFund entity + APIs, MFAPI scheduler, NAV history	Add AMC page, Add Fund page, Browse Funds page with filters
Week 3	Investment + Portfolio	Investment entity, units calculation, Portfolio entity, returns calculation	Add Investment page, Portfolio page with live returns
Week 4	SIP Management	Sip entity + APIs, SipTransaction entity, monthly scheduler	My SIPs page — Active, History and Missed tabs
Week 5	Admin + Users	Admin dashboard API, User management APIs, profile and SIP view	Admin Dashboard, Users Management page, investor profile view
Week 6	Alert System	Alert entity, template engine, automated and manual alert APIs	Send Alerts page (admin), Notification bell (investor)
Week 7	Investor Dashboard	Dashboard summary API aggregating all investor data	Investor Dashboard — metric cards, portfolio summary, upcoming SIPs
Week 8	Testing + Polish	Bug fixes, edge case handling, API testing in Postman	UI polish, loading states, error messages, demo preparation

14. Limitations

7. NAV data is fetched from a free public API (MFAPI) — not a certified financial data source.
8. No real payment gateway integration — SIP payments are recorded manually.
9. Admin is a single pre-configured user — no admin registration or multi-admin support.
10. No email or SMS notification integration — all alerts are in-app only.
11. No email verification on investor registration.
12. SIP auto-deduction is simulated — no actual bank deduction.

15. Future Enhancements

13. Email and SMS notifications using SMTP or Twilio API integration.
14. SIP auto-deduction simulation using monthly Spring scheduler.
15. NAV history line charts per fund showing price trends over time.
16. Goal-based SIP calculator — enter target amount and get recommended SIP.
17. Export portfolio as PDF report for investor.
18. Migration to microservices architecture with API Gateway and Eureka.
19. Mobile application using React Native.